FULL-STACK WEB DEVELOPMENT

Building a Full-Stack Mini App

Learn how React talks to Flask one step at a time through a Study Tip Generator

1. What Are We Building & Architecture Overview

We are constructing a client-server mini application where a student enters their name on a React frontend interface, sends an asynchronous HTTP POST request, and receives a personalized study tip generated by a Flask backend.

1	2	3	4	5
React UI Student types name in input field	**Fetch() POST** Sends JSON object to Flask endpoint	**Flask API** Processes request and builds tip string	**JSON Response** Returns {"tip": "..."}	**React Display** Renders study tip on UI

2. Project Structure & Prerequisites

To run this full-stack application, ensure you have **Python 3**, **Node.js**, and a code editor (like VS Code) installed.

Organize your project folder with separate directories for backend and frontend:

```
my-app/
├── backend/
│   ├── app.py          # Flask backend server script
│   └── requirements.txt
└── frontend/
    ├── package.json
    └── src/
        └── App.js      # React main component
```

Environment Setup Commands:

```
# Backend Setup
cd backend
pip install flask flask-cors

# Frontend Setup
cd frontend
npx create-react-app .
npm start
```

3. Step 1: Building the Flask Backend

Flask is a lightweight Python web framework used here to expose an API endpoint (`/api/tip`).

```python
# File: backend/app.py
from flask import Flask, request, jsonify
from flask_cors import CORS

app = Flask(__name__)
CORS(app)  # Enables Cross-Origin Resource Sharing

@app.route('/api/tip', methods=['POST'])
def get_tip():
    data = request.get_json()
    name = data.get('name', 'Student')
    tip = f"Hey {name}, try the Pomodoro method today!"
    return jsonify({'tip': tip})

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

Key Backend Concepts:

- **Route & Method:** `@app.route('/api/tip', methods=['POST'])` defines the URL path and restricted HTTP verb.
- **JSON Processing:** `request.get_json()` extracts incoming payload data, and `jsonify()` serializes Python dictionaries to JSON format.
- **CORS Support:** `flask-cors` permits requests from React running on port 3000 to Flask on port 5000, avoiding browser security blocks.

4. Testing Your API Independently

Before connecting React, verify your backend using `curl` or Postman:

```
$ python app.py
* Running on http://127.0.0.1:5000

# Execute Test Request:
curl -X POST http://127.0.0.1:5000/api/tip   -H "Content-Type: application/json"   -d '{"name": "A

# Expected Output (200 OK):
{"tip": "Hey Asha, try the Pomodoro method today!"}
```

5. Step 2: Building the React Frontend

The React UI handles user interaction, state management, and API calls using the native JavaScript `fetch` function.

```
// File: frontend/src/App.js
import React, { useState } from 'react';

function App() {
  const [name, setName] = useState('');
  const [tip, setTip] = useState('');

  const handleSubmit = async (e) => {
    e.preventDefault();
    const response = await fetch('http://127.0.0.1:5000/api/tip', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ name })
    });
    const data = await response.json();
    setTip(data.tip);
  };

  return (
    <div style={{ padding: '20px' }}>
      <h2>Study Tip Generator</h2>
      <form onSubmit={handleSubmit}>
        <input
          type="text"
          value={name}
          onChange={(e) => setName(e.target.value)}
          placeholder="Enter your name..."
        />
        <button type="submit">Get My Tip</button>
      </form>
      {tip && <p style={{ marginTop: '15px' }}>{tip}</p>}
    </div>
  );
}

export default App;
```

6. Troubleshooting & Next Steps

Common Troubleshooting Checklist:

- **Flask Server Not Running:** Ensure `python app.py` is active in the terminal before submitting from React.
- **CORS Blocked Error:** Verify `CORS(app)` is initialized in Flask to allow origin `http://localhost:3000`.
- **Network 404 / Port Mismatch: Confirm fetch URL specifies port `5000` and path `/api/tip`.**

Student Practice Worksheet

Topic: Full-Stack React & Flask Integration | Assessment

Student Name: _____ **Date:** _____

Question 1: Conceptual Architecture

Explain the role of the frontend and backend in a full-stack web application. What specific tasks does React perform versus Flask in the Study Tip Generator project?

Question 2: HTTP Protocols & Methods

Why was an HTTP POST request used for fetching the study tip instead of an HTTP GET request? What key information is included in the POST request body?

Question 3: Cross-Origin Resource Sharing (CORS)

Suppose React runs on `http://localhost:3000` and Flask runs on `http://localhost:5000`. What error will the browser display if `flask-cors` is not enabled, and why does the browser enforce this policy?

Question 4: Code Analysis & Debugging

Examine the snippet below from React. Identify two errors that will prevent the study tip from properly displaying:

```
const handleSubmit = async (e) => {
  const response = fetch('http://127.0.0.1:5000/api/tip', {
    method: 'GET',
    body: { name: name }
  });
  setTip(response.tip);
};
```

Question 5: Feature Expansion Challenge

Write out the Python code modification for `app.py` if the frontend sent both a student's `name` and their chosen `subject` (e.g., "Math" or "History") to generate a subject-specific study tip.